

Reto: Crea una clase `Cadena` que haga las mismas operaciones que la clase `string`: + concatena, = asigna, += añade, == compara y `c_str()` para convertir la `Cadena` en un `char *`. Además, debe poder crear y asignar a una `Cadena` otra `Cadena` o una cadena de caracteres `char *`. También debe ser capaz de utilizar `cout <<` y `cin >>` para mostrar la `Cadena` por pantalla y leerla desde teclado y debe poder usar `getline()` para leer cadenas compuestas (frases).

Haz un `main()` que pruebe su funcionamiento y luego implementa el ejemplo de la pág. 16

La clase `string` permite trabajar con cadenas de forma cómoda: no es necesario indicar la longitud de la cadena como ocurre en C, sino que esta se asigna automáticamente al leer mediante `cin >>`, tampoco hay que preocuparse por reservar o liberar memoria o por que se desborde la cadena.

La clase `Cadena` que vamos a implementar debe ser capaz de:

- crear una `Cadena`: a) vacía, b) a partir de una cadena de caracteres (`char*`), c) a partir de otra `Cadena`
- no nos tenemos que preocupar de liberar la memoria: d)
- convertir la `Cadena` en una cadena de caracteres (`char *`): e)
- asignarle a una `Cadena` con `operator=`: g) otra `Cadena`, f) una cadena de caracteres (`char*`)
- añadirle a una `Cadena` con `operator+=`: i) otra `Cadena`, h) una cadena de caracteres (`char*`)
- concatenar con `operator+`: k) `Cadena` con `Cadena`, j) `Cadena` con `char*`, m) `char*` con `Cadena`
- comparar con `operator==`: o) `Cadena` con `Cadena`, n) `Cadena` con `char*`, p) `char*` con `Cadena`
- Debe ser capaz de utilizar el `operator<<` para poder mostrar la `Cadena` con un `cout`: q)
- Debe ser capaz de utilizar el `operator>>` para poder crear una `Cadena` con un `cin` desde teclado: r)
- Debe poder utilizar la función `getline()` para poder leer una frase/cadena con espacios: s)
- El comportamiento del `cin >>` y del `getline()` en `Cadena` debe ser idéntico al del tipo `string`

SOLUCION:

```
#include <iostream>
#include <cstring>
using namespace std;

class Cadena {
    char *cad;
public:
    Cadena() { cad=new char[1]; strcpy(cad,""); } // a)
    Cadena(const char *c) { cad=new char[strlen(c)+1]; strcpy(cad,c); } // b)
    Cadena(const Cadena &c) { cad=new char[strlen(c.cad)+1]; strcpy(cad,c.cad); } // c)
    virtual ~Cadena() { delete [] cad; } // d)

    const char* c_str() const { return cad; } // e)

    Cadena& operator=(const char *c) { // f)
        delete [] cad; cad=new char[strlen(c)+1]; strcpy(cad,c);
        return *this;
    }

    Cadena& operator=(const Cadena &c) { // g)
        if (this!=&c) {
            delete [] cad; cad=new char[strlen(c.cad)+1]; strcpy(cad,c.cad);
        }
        return *this;
    }

    Cadena& operator+=(const char *c) { // h)
        char *aux=cad;
        cad=new char[strlen(cad)+strlen(c)+1];
        strcpy(cad,aux); strcat(cad,c);
        delete [] aux;
        return *this;
    }

    Cadena& operator+=(const Cadena &c) { // i)
        return operator+=(c.cad); // llama a h)
    }

    Cadena operator+(const char *c) const { // j)
        char *aux=new char[strlen(cad)+strlen(c)+1];
        strcpy(aux,cad); strcat(aux,c);
        Cadena local(aux);
        delete [] aux;
        return local;
    }

    Cadena operator+(const Cadena &c) const { // k)
        return operator+(c.cad); // llama a j)
    }

    bool operator==(const char *c) const { return strcmp(cad,c)==0; } // n
```

```

    bool operator==(const Cadena &c) const { return strcmp(cad,c.cad)==0; } // o)
};

Cadena operator+(const char *s, const Cadena &c) { // m)
    return Cadena(s)+c; //llama a k)
}

bool operator==(const char *s, const Cadena &c) { // p)
    return strcmp(s, c.c_str())==0;
}

ostream &operator<<(ostream &s, const Cadena &c) { // q)
    s << c.c_str();
    return s;
}

istream &operator>>(istream &s, Cadena &c) { // r)
    char aux[200];
    s >> aux;
    c=aux; //llama a f)
    return s;
}

istream& getline(istream& is, Cadena& c, char delim='\n') { // s)
    char aux[200];
    is.getline(aux, 200, delim);
    c=aux; //llama a f)
    return is;
}

int main() {
    Cadena c1="Ana", c3=" Lopez", c4=c1, c5;
    const Cadena c2=" Maria";
    cout << c1 << c2 << " Perez" << c3 << endl;
    if (c5==" " && c1=="Ana" && "Ana"==c1 && c1==c4)
        cout << "c5 esta en blanco, c1 y c4 son iguales: " << c4 << endl;
    c1=c1;
    c5=c3;
    if (c5==c3)
        cout << "'" << c5 << "' igual a '" << c3.c_str() << "'\n";
    if (!(c2==c1))
        cout << "'" << c2.c_str() << "' distinto a '" << c3 << "'\n";
    cout << c1 << c5 << c3 << endl;
    c3=" Molina";
    cout << c1.c_str() << c5 << c3.c_str() << endl;
    c1="Jose";
    c1=c1+c2+" Roldan"+c3;
    cout << c1 << " termina en" << c3.c_str() << endl;
    c5="Pedro" + c5 + " Marquez";
    cout << c5 << endl;
    cout << "Indica 2 palabras: ";
    cin >> c1 >> c5;
    cout << "Las palabras son: \n";
    cout << c1 << endl << c5 << endl;
    cout << "Introduce 2 nombres compuestos: " << endl;
    cin.ignore(); getline(cin,c1); getline(cin, c5);
    cout << "Los nombres introducidos son: \n";
    cout << c1 << endl << c5 << endl;
    cout << c1 << " tiene " << strlen(c1.c_str()) << " caracteres" << endl;

    c1=c3=c4="luis perez";
    cout << c1 << " - " << c3 << " - " << c4 << endl;
    c3="juan";
    cout << c1 << " - " << c3 << " - " << c4 << endl;

    system("PAUSE");
    return 0;
}

```

SALIDA POR PANTALLA

```

Ana Maria Perez Lopez
c5 esta en blanco, c1 y c4 son iguales: Ana
' Lopez' igual a ' Lopez'
' Maria' distinto a ' Lopez'
Ana Lopez Lopez
Ana Lopez Molina
Jose Maria Roldan Molina termina en Molina
Pedro Lopez Marquez
Indica 2 palabras: perro gato
Las palabras son:
perro
gato
Introduce 2 nombres compuestos:
juan antonio
jose enrique
Los nombres introducidos son:
juan antonio
jose enrique
juan antonio tiene 12 caracteres
luis perez - luis perez - luis perez
luis perez - juan - luis perez

```

Ejemplo de la pág. 16 implementado con la clase Cadena hecha por nosotros vs string

Versión C++ (Cadena)	Versión C++ (string)
<pre> #include <iostream> //cin, cout #include <cstring> //strlen(), strcpy(), strcmp() #include "Cadena.h" //clase Cadena using namespace std; class Persona { Cadena nombre; const int edad; //la edad no se puede modificar public: Persona(Cadena nom, int e); //Persona(const Persona& p); //NO NECESARIO //virtual ~Persona(); //NO NECESARIO Persona& operator=(const Persona& p); int getEdad() const { return this->edad; } Cadena getNombre() const { return this->nombre; } void setNombre(Cadena nom); bool operator==(Persona p) const; }; Persona::Persona(Cadena nom, int e): edad(e) { this->nombre=nom; //this->edad=e; //ERROR edad es constante } /* Persona::~Persona() { //NO HAY QUE HACER NADA } Persona::Persona(const Persona& p) : edad(p.edad) { this->nombre=p.nombre; } */ //necesario porque edad no puede cambiar Persona& Persona::operator=(const Persona& p) { if (this != &p) { this->nombre=p.nombre; } return *this; } void Persona::setNombre(Cadena n) { this->nombre=n; } bool Persona::operator==(Persona p) const { return (this->edad==p.edad && this->nombre=p.nombre;); } ostream& operator<<(ostream &s, const Persona &p) { s << p.getNombre() << " (" << p.getEdad() << ")"; return s; } </pre>	<pre> #include <iostream> //cin, cout #include <string> //string using namespace std; class Persona { string nombre; const int edad; //la edad no se puede modificar public: Persona(string nom, int e); //Persona(const Persona& p); //NO NECESARIO //virtual ~Persona(); //NO NECESARIO Persona& operator=(const Persona& p); int getEdad() const { return this->edad; } string getNombre() const { return this->nombre; } void setNombre(string nom); bool operator==(Persona p) const; }; Persona::Persona(string nom, int e): edad(e) { this->nombre=nom; //this->edad=e; //ERROR edad es constante } /* Persona::~Persona() { //NO HAY QUE HACER NADA } Persona::Persona(const Persona& p) : edad(p.edad) { this->nombre=p.nombre; } */ //necesario porque edad no puede cambiar Persona& Persona::operator=(const Persona& p) { if (this != &p) { this->nombre=p.nombre; } return *this; } void Persona::setNombre(string n) { this->nombre=n; } bool Persona::operator==(Persona p) const { return (this->edad==p.edad && this->nombre=p.nombre;); } ostream& operator<<(ostream &s, const Persona &p) { s << p.getNombre() << " (" << p.getEdad() << ")"; return s; } </pre> <pre> int main() { Persona p1("Jose Pepe", 25), p2=p1; p1.setNombre("Luis Maria"); if (p1==p2) cout << "p1 y p2 son iguales\n"; else cout << "p1 y p2 NO son iguales\n"; cout << p1 << " y " << p2 << " tienen " << p1.getEdad() << " tacos\n"; cout << "p1 es " << p1.getNombre() << endl; p2=p1; if (p1==p2) cout << "p1 y p2 ahora son iguales\n"; cout << p1 << " y " << p2 << endl; system("PAUSE"); return 0; } </pre> PANTALLA: <pre> p1 y p2 NO son iguales Luis Maria (25) y Jose Pepe (25) tienen 25 tacos p1 es Luis Maria p1 y p2 ahora son iguales Luis Maria (25) y Luis Maria (25) Presione una tecla para continuar ... </pre>

Como podemos observar el código de los métodos es el mismo y el resultado por pantalla también